

Assignment No - 01 (Group A)

Problem statement:

Data Wrangling- I

Perform the following operations using Python on any open source dataset (e.g., data.csv)

1. Import all the required Python Libraries.
2. Locate an open source data from the web (e.g. <https://www.kaggle.com>). Provide a clear description of the data and its source (i.e., URL of the web site).
3. Load the Dataset into pandas data frame.
4. Data Pre-processing: check for missing values in the data using pandas `isnull()`, `describe()` function to get some initial statistics. Provide variable descriptions. Types of variables etc. Check the dimensions of the data frame.
5. Data Formatting and Data Normalization: Summarize the types of variables by checking the data types (i.e., character, numeric, integer, factor, and logical) of the variables in the data set. If variables are not in the correct data type, apply proper type conversions.
6. Turn categorical variables into quantitative variables in Python. In addition to the codes and outputs, explain every operation that you do in the above steps and explain everything that you do to import/read/scrape the data set.

Pre-requisite

- Python programming concepts

Objective

1. Should be able to make raw data useful by applying scientific data processing libraries.
2. Understand and apply the python libraries such as numpy, pandas, matplotlib etc.

Software and Hardware requirements:-

1. **Operating system:** Linux- Ubuntu 16.04 to 17.10, or Windows 7 to 10,
2. **RAM-** 2GB RAM (4GB preferable)
3. **IDE :-** Anaconda Jupiter Notebook / pycharm / Visual Studio

Theory-

Data wrangling

Data wrangling is the process of cleaning and unifying messy and complex data sets for easy access and analysis.

With the amount of data and data sources rapidly growing and expanding, it is getting increasingly essential for large amounts of available data to be organized for analysis. This process typically includes manually converting and mapping data from one raw form into another format to allow for more convenient consumption and organization of the data

The Goals of Data Wrangling

- Reveal a "deeper intelligence" by gathering data from multiple sources
- Provide accurate, actionable data in the hands of business analysts in a timely matter
- Reduce the time spent collecting and organizing unruly data before it can be utilized
- Enable data scientists and analysts to focus on the analysis of data, rather than the wrangling
- Drive better decision-making skills by senior leaders in an organization

Key Steps to Data Wrangling

1. **Data Acquisition:** Identify and obtain access to the data within your sources.
2. **Joining Data:** Combine the edited data for further use and analysis.
3. **Data Cleansing:** Redesign the data into a usable and functional format and correct/remove any bad data.

Methods and function used:-

1. import panda as pd:-

pd.read_csv():-

CSV files are the Comma Separated Files. It allows users to load tabular data into a DataFrame, which is a powerful structure for data manipulation and analysis.

To access data from the CSV file, we require a function read_csv() from Pandas that retrieves data in the form of the data frame.

Syntax: `pd.read_csv(filepath_or_buffer, sep=',', header='infer', index_col=None, usecols=None, engine=None, skiprows=None, nrows=None)`

Parameters:

filepath_or_buffer: Location of the csv file. It accepts any string path or URL of the file.

sep: It stands for separator, default is ‘, ‘.

header: It accepts int, a list of int, row numbers to use as the column names, and the start of the data. If no names are passed, i.e., header=None, then, it will display the first column as 0, the second as 1, and so on.

usecols: Retrieves only selected columns from the CSV file.

nrows: Number of rows to be displayed from the dataset.

index_col: If None, there are no index numbers displayed along with records.

skiprows: Skips passed rows in the new data frame.

2. data_frame.head()

Return the first 5 rows of the DataFrame. The head() method returns the first 5 rows if a number is not specified.

Syntax `dataframe.head(n)`

A DataFrame with headers and the specified number of rows.

3. dataframe.shape

Return a tuple representing the dimensionality of the Data Frame.

Ex. An output(48,14) representing 48 rows and 14 columns

4. Conversion with convert_dtypes

The convert_dtypes DataFrame method will convert all columns to the best possible types that support pd.NA, the Pandas missing value.

Note that this method will not convert numeric strings to integers or floats.

Change all columns to the appropriate types

```
df.convert_dtypes()
```

```
# Change column B and C's values to a numeric type
```

```
df[['B', 'C']] = df[['B', 'C']].apply(pd.to_numeric)
```

```
# Change column B and C's values to integers
```

```
df = df.astype({'B': int, 'C': int})
```

5. replace() method

Replacing is one of the methods to convert categorical terms into numeric. For example, We will take a dataset of people's salaries based on their level of education. This is

an ordinal type of categorical variable. We will convert their education levels into numeric terms.

Syntax:

```
replace(to_replace=None, value=None, inplace=False, limit=None, regex=False, method='pad')
```

```
df['Education'].replace(['Under-Graduate', 'Diploma '], [0, 1], inplace=True)
```

6.DataFrame. describe()

The describe() method returns description of the data in the DataFrame.

If the DataFrame contains numerical data, the description contains these information for each column:

count - The number of not-empty values.

mean - The average (mean) value.

std - The standard deviation.

min - the minimum value.

25% - The 25% percentile*.

50% - The 50% percentile*.

75% - The 75% percentile*.

max - the maximum value.

*Percentile meaning: how many of the values are less than the given percentile

7.isnull()

Replace all values in the DataFrame with True for NULL values, otherwise False:

Syntax

```
dataframe.isnull()
```

8. df.shape

```
# check out the dimension of the dataset
```

9. df.dtypes

```
# Look at the data types for each column Do these match the expected data types for the columns?
```

10. **df.head()**

read the first five rows

11. **df.tail()**

read the Last five rows

12. **df.columns.values**

return an array of column names

13. **df.columns.values.tolist()**

return a List of column names

Check Missing Values

df.isnull()

Checking missing **Values**

Returns True if the value is **NaN / missing**

Returns False if the value is **present**

df.notnull()

Checking non-missing **Values**

Opposite of isnull()

Returns True for **non-missing values**

Returns False for **missing values**

df.isnull().values.any()

Check if there's missing **Values**

.values → converts DataFrame to NumPy array

.any() → checks if **at least one True exists**

True → missing values exist

False → no missing values

df.isnull().sum().sum()

To get the total number of missing values in the entire dataset

First sum() → counts missing values column-wise

Second sum() → adds all missing values across columns

df.isnull().sum()

#Check missing by variable

#Counts missing values for each column individually

```
df[df["height"].notnull()] #return rows where height is not null
```

Get Information about Missing Values

```
df[df["height"].notnull()]
```

#return rows where height is not null

```
df["height"].notnull()
```

creates a boolean mask

Filters rows where **height is NOT missing**

Returns only rows with valid height values

Purpose:

To **remove rows with missing values in a specific column**

Result:

A filtered DataFrame with **no missing values in height**

Fill In Missing Values

For quantitative variables, we may replace missing values with the sample mean, mode, median, or other numbers. For categorical variables, we can create a new category for missing values by replacing missing values with a string.

Replace missing values with 0.

```
Fill_no = df.fillna(0)
```

Fill in missing with 0 and save the data to Fill_no

Replaces **all missing values (NaN)** in the DataFrame df with **0**

Stores the cleaned DataFrame in a new variable called Fill_no

The original DataFrame df remains **unchanged**

```
df['DataFrame Column'] = df['DataFrame Column'].fillna(0)
```

Fill in missing for a singular column

```
df['DataFrame Column']
```

Selects a **specific column** from the DataFrame.

fillna(0)

Replaces all **missing (NaN) values** in that column with 0.

df['DataFrame Column'] = ...

Assigns the updated column **back to the DataFrame**, making the change permanent

Fill_no.head()

df.fillna("missing")

Replace all missing with string missing.

#**Fill** in missing with a string: -missing- and save the data to **Fill_str**

Replaces **all missing values (NaN)** in the DataFrame df with the **string "missing"**.

Useful mainly for **categorical or text data**.

df["Height"].fillna(df["Height"].mean(), inplace=True)

Fill missing values with the sample mean

df["Height"]

Selects the **Height** column from the DataFrame.

mean()

Calculates the **average (mean)** of all **non-missing values** in the Height column.

fillna(df["Height"].mean())

Replaces all **missing (NaN) values** in the Height column with the **mean value** of that column.

inplace=True

Applies the change **directly to the original DataFrame** without creating a new copy.

Dropping Data

We may want to drop duplicate rows if any and save the changes to the original data.

df.drop_duplicates(inplace=True)

We also may want to drop some observations or some columns.

```
df = df.drop(df.index([1,2,3]))
```

```
# Droa the first second and third row
```

```
df = df.drop(df.index[range(1,11)])
```

```
# drop the 2nd to the 11th row
```

```
df = df.drop(["Height"], axis = 1)
```

```
# drop the Height column from the data
```

```
df = df.drop(["Height", "Season"], axis =1)
```

```
#Drop the height and season columns from the dataset
```


Conclusion:

In this way we have explored the functions of the python library for Data Preprocessing, Data Wrangling Techniques and How to handle missing values.